

**DEPARTMENT OF ENGINEERING AND TECHNOLOGY
GURU NANAK DEV UNIVERSITY REGIONAL CAMPUS JALANDHAR
(PUNJAB) INDIA – 144007**

SIX MONTH INDUSTRIAL TRAINING

CSD 481 – FINAL TERM REPORT

ON

"FEAST FLOW"

Submitted in partial fulfillment of the requirement for award of Degree of

BACHELOR OF TECHNOLOGY

(COMPUTER SCIENCE AND ENGINEERING)

SESSION: 2022–26

SUBMITTED TO:

DR. SHEETAL KALRA

Professor

Guru Nanak Dev University

Regional Campus, Jalandhar

SUBMITTED BY:

AERONBIR SINGH BRAR

B.Tech CSE 8th Sem

Roll No. 17032249011

**DEPARTMENT OF ENGINEERING AND TECHNOLOGY
GURU NANAK DEV UNIVERSITY REGIONAL CAMPUS JALANDHAR
(PUNJAB) INDIA – 144007**

INDEX

Sr. No.	Title	Page No.
Front Matter		
—	Declaration	i
—	Certificate	ii
—	Acknowledgement	iii
—	Certificate from the Organisation	iv
1	Introduction to Project 1.1 Introduction & Scope of Project 1.2 Description of Project 1.3 Objectives of the Project	1 – 6
2	Requirement Analysis and Literature Survey 2.1 Existing Projects of Similar Characteristics 2.2 Need to Develop this Project 2.3 Course of Action 2.4 Literature Survey	7 – 14
3	Planning of the Project 3.1 Objectives of the Project 3.2 Planned Steps to Achieve above Objectives 3.3 System Architecture Planning	15 – 20
4	Hardware and Software Requirements 4.1 Hardware Requirements 4.2 Software Requirements 4.3 Tools Used for Development 4.4 Techniques Used for Development	21 – 27
5	Design of the Project 5.1 ER/DFD/Flow Chart Diagrams 5.2 Class Diagram 5.3 Data Storage Tables	28 – 35
6	Implementation of the Project 6.1 Screenshots of Working Project 6.2 Code Snippets (6.2.1 – 6.2.13)	36 – 47

Sr. No.	Title	Page No.
7	Conclusions / Future Scope 7.1 Conclusions 7.2 Results in Tabular Form 7.3 Future Scope	48 – 51
8	About the Organisation	52 – 53
9	References	54 – 55

DECLARATION

I, "Aeronbir Singh Brar", declare that the work presented in this project titled "**Feast Flow**", submitted to the Department of Engineering & Technology, Guru Nanak Dev University Regional Campus, Jalandhar for the award of degree of Bachelor of Technology in Computer Science and Engineering, is my original work. I have not plagiarized or submitted the same work for the award of any other degree. In case this undertaking is found incorrect, I accept that my degree may be unconditionally withdrawn.

May, 2026

Aeronbir Singh Brar
Roll No. 17032249011

CERTIFICATE

Certified that the work contained in this report of project titled "**Feast Flow**", by **Aeronbir Singh Brar**, has been carried out under my supervision.

Dr. Sheetal Kalra

(Supervisor)

Professor

Guru Nanak Dev University

Regional Campus, Jalandhar

Er. Neena Madan

(Training Incharge & Class Coordinator)

Assistant Professor

Guru Nanak Dev University

Regional Campus, Jalandhar

ACKNOWLEDGEMENT

I, "Aeronbir Singh Brar", am highly grateful to Dr. Deep Kamal Kaur Randhawa, Professor & Head, Department of Engineering and Technology, Guru Nanak Dev University Regional Campus, Jalandhar, for providing this opportunity to carry out the present report/work. The consistent guidance and encouragement received from Dr. Sheetal Kalra, our project Faculty Mentor, has been of great help in carrying out the present work and is acknowledged with reverential thanks. Without her wise counsel and able guidance, it would have been difficult to complete the report in this manner.

Aeronbir Singh Brar
Roll No. 17032249011

CERTIFICATE FROM THE ORGANIZATION

(Attached separately)

CHAPTER 1

INTRODUCTION TO PROJECT

1.1 Introduction & Scope of Project

In recent years, the rapid growth of internet technologies and digital platforms has significantly transformed the way businesses operate across various industries. One of the sectors that has experienced a major shift is the food and restaurant industry. Traditionally, restaurants relied on manual processes such as dine-in services, handwritten orders, and telephonic bookings. While these methods served their purpose in the past, they are no longer sufficient to meet the demands of modern customers. Today's consumers expect convenience, speed, and accessibility. With the widespread use of smartphones and the internet, customers prefer ordering food online rather than visiting restaurants physically or placing phone calls. This shift has led to the rise of digital food ordering systems and e-commerce platforms tailored specifically for restaurants.

The *Restaurant E-Commerce Website with Intelligent Order Management* is designed as a full-stack web application that aims to digitalize and automate the restaurant ordering process. The system provides a centralized online platform where customers can explore menus, select food items, manage their cart, and place orders efficiently. At the same time, it provides restaurant administrators with tools to manage menu items, track orders, and maintain customer data.

Need for Digital Transformation in Restaurants

The traditional restaurant management system suffers from several limitations:

- High dependency on manual labor
- Increased chances of order miscommunication
- Long waiting times during peak hours
- Lack of proper record-keeping
- Limited accessibility for customers

To overcome these challenges, a digital system is necessary. The proposed project addresses these issues by introducing automation, accuracy, and efficiency into restaurant operations.

Scope of the Project

The scope of this project covers the design, development, and deployment of a web-based restaurant ordering system. It includes the following major components:

1. User Interface Development

- Development of a responsive and user-friendly interface
- Ensuring compatibility across devices (desktop, tablet, mobile)
- Designing intuitive navigation for better user experience

2. Backend System Development

- Creation of RESTful APIs for handling business logic
- Implementation of server-side operations using Node.js and Express.js
- Handling user authentication, order processing, and data management

3. Data Storage Management

- Use of JSON file-based storage for storing application data
- Efficient storage of user information, menu item details, orders, and cart data

4. Security Implementation

- Secure login system
- Password encryption
- Role-based access control (admin vs user)

5. Testing and Deployment

- Testing system functionality
- Debugging errors
- Deploying application to a live environment

Limitations of the Project

- Does not include live GPS-style delivery tracking
- Online payment gateway integration is not implemented; checkout supports cash/card on delivery
- Limited to single restaurant system
- Does not include AI-based recommendations (future enhancement)

Future Scope of the Project

- Integration of online payment systems (Paytm, Google Pay)
- Implementation of admin-managed order status updates
- Development of mobile applications (Android/iOS)
- Addition of AI-based food recommendation systems
- Expansion to a multi-restaurant platform

1.2 Description of Project

The Restaurant E-Commerce Website is a web-based platform that facilitates interaction between customers and restaurant administrators through an online interface. The system is designed using modern web technologies and follows a client-server architecture.

Overview of System Functionality

The system operates through two main modules:

1. Customer Module

The customer module provides an interactive and easy-to-use interface for users. Key Features:

- **User Registration and Login:** Users can create accounts and log in securely.
- **Menu Browsing:** Users can view food items categorized into sections such as snacks, beverages, main course, etc.
- **Menu Item Details View:** Each item includes Name, Price, Description, and Image.
- **Cart Management:** Add items, remove items, update quantities.
- **Order Placement:** Users can place orders through a simple checkout process.
- **Order History:** Users can track their previous orders.

2. Admin Module

The admin module provides full control over the system. Key Features:

- **Menu Management:** Add, update, and delete food items.
- **Category Management:** Organize menu items into categories.
- **Order Management:** View all orders and update order status.

- **System Monitoring:** Track system activity, reservations, contacts, and newsletter subscribers.

System Architecture

The system follows a three-tier architecture, which ensures separation of concerns:

1. **Presentation Layer (Frontend)** – Built using HTML5, CSS3, and JavaScript; handles user interface and interactions.
2. **Application Layer (Backend)** – Built using Node.js and Express.js; handles business logic and API requests.
3. **Data Layer (Data Storage)** – JSON files are used for storing data; provides fast and flexible data retrieval.

Working of the System

1. User logs into the system
2. User browses menu items
3. User adds items to cart
4. User places order
5. Order is stored in data storage
6. Admin views and processes order

1.3 Objectives of the Project

The objectives of this project are defined using the SMART framework to ensure clarity and effectiveness.

1.3.1 Specific Objectives

- To develop a complete restaurant e-commerce system
- To automate manual ordering processes
- To create a user-friendly interface
- To implement secure login and authentication

1.3.2 Measurable Objectives

- Reduce order processing errors by at least 70%

- Improve customer satisfaction through faster service
- Ensure system uptime and reliability

1.3.3 Achievable Objectives

- Use HTML/CSS/JavaScript and Node.js/Express technologies for efficient development
- Utilize existing frameworks and tools
- Implement features within available resources

1.3.4 Relevant Objectives

- Aligns with current industry trends
- Solves real-world restaurant problems
- Enhances digital transformation

1.3.5 Time-Bound Objectives

The project is planned to be completed within six months:

- Month 1: Requirement analysis and planning
- Month 2: System design
- Month 3–4: Development
- Month 5: Testing
- Month 6: Deployment

Learning Outcomes

- Full-stack web development
- API design and integration
- JSON data management
- Debugging and testing
- Version control using Git

CHAPTER 2

REQUIREMENT ANALYSIS AND LITERATURE SURVEY

2.1 Existing Projects of Similar Characteristics

Before designing the proposed system, a detailed study of existing systems and similar projects was conducted. This analysis helps in understanding current solutions, identifying their limitations, and designing a better system.

2.1.1 Traditional Restaurant Systems

Traditional restaurant systems rely heavily on manual processes such as taking orders verbally or on paper, manual billing, and direct communication between customer and staff. Limitations include high error rates, time-consuming processes during peak hours, no data storage for customer preferences, limited accessibility, and inefficient management.

2.1.2 Third-Party Food Delivery Platforms

Platforms such as Swiggy, Zomato, and Uber Eats have revolutionized food ordering. While they offer easy online ordering and wide customer reach, they impose high commission charges (20–30%), reduce profit margins, create dependency on third-party services, and restrict customer data access.

2.1.3 Existing E-Commerce Website Templates

Generic e-commerce templates lack customization for restaurant workflows, have no optimized order processing logic, limited scalability, and basic UI/UX.

Conclusion: The analysis clearly shows that traditional systems lack efficiency, third-party platforms reduce business control, and generic solutions lack specialization. Therefore, there is a need for a custom-built, independent restaurant system.

2.2 Need to Develop this Project

2.2.1 Operational Need

- Automate manual processes
- Reduce human errors
- Improve order accuracy

- Speed up service

2.2.2 Business Need

- Eliminate dependency on third-party platforms
- Avoid commission charges
- Improve profit margins
- Maintain direct customer relationship

2.2.3 Customer Need

- Convenience of ordering from anywhere
- Faster service
- Better user experience
- Access to order history

2.2.4 Technological Need

- Adoption of modern technologies
- Centralized data management
- Scalable system architecture

2.2.5 Competitive Need

- Restaurants need digital presence
- Customers prefer online ordering
- Faster systems attract more users

2.3 Course of Action

The development of the system follows a structured and systematic approach:

- **Step 1 – Requirement Analysis:** Identify system users (Customer, Admin), define functionalities, understand constraints.
- **Step 2 – System Design:** Design system architecture, define modules and components, plan data flow.
- **Step 3 – Data File Design:** Identify entities (Users, Menu Items, Orders), define relationships.

- **Step 4 – Backend Development:** Setup Node.js server, develop REST APIs, implement authentication.
- **Step 5 – Frontend Development:** Design UI using JavaScript, create reusable sections and helpers, integrate APIs.
- **Step 6 – Integration:** Connect frontend with backend API routes, handle data exchange and errors.
- **Step 7 – Testing:** Functional, API, and UI testing.
- **Step 8 – Deployment:** Deploy application, monitor performance.

Development Methodology: The project follows an iterative development approach, where features are developed in phases, continuous testing is performed, and improvements are made incrementally.

2.4 Literature Survey

JOURNAL

[1] A. Kumar and S. Gupta, "Secure Role-Based Access Control in Web Applications," *International Journal of Computer Applications*, vol. 183, no. 12, pp. 25–31, 2021.

This paper discusses the implementation of Role-Based Access Control (RBAC) mechanisms in modern web applications. The authors explain how authentication and authorization layers improve system security and prevent unauthorized data access. They highlight the use of JWT tokens and middleware-based validation for maintaining secure sessions. The study also emphasizes password hashing and encrypted communication protocols.

This research helped in understanding secure authentication systems and implementing role-based access control within the Restaurant E-Commerce Website.

ONLINE JOURNAL

[2] N. Singh and A. Kumar. (2022). RESTful API Design Patterns for Secure Web Applications. *International Journal of Web Engineering and Technology*. [Online]. Available: <https://ieeexplore.ieee.org>

This online journal article explores RESTful API design patterns for building secure and scalable web applications. The authors explain how stateless HTTP communication,

structured API endpoints, and JSON-based data exchange improve reliability and maintainability of e-commerce systems. The study also discusses the use of the Fetch API for asynchronous client-server communication and how middleware-based request handling improves error management.

The findings of this study influenced the design of RESTful API endpoints and the structured use of the Fetch API for communication between the frontend and backend of the Restaurant E-Commerce Website.

BOOKS

[3] U.K. Roy, *Web Technologies*.

This book provides foundational knowledge of HTML, CSS, JavaScript, and modern web technologies. It explains client-server communication, webpage structuring, responsive design, and scripting techniques. The concepts helped in designing the frontend structure and responsive user interface of the Restaurant E-Commerce Website.

[4] D. Herron, *Node.js Web Development*.

This book explains backend development using Node.js and covers server-side programming, asynchronous operations, routing, and REST API development. It also discusses middleware integration and scalable backend architecture. The concepts helped in developing the backend server and APIs for the Restaurant E-Commerce Website.

[5] A. Banks and E. Porcello, *Learning JavaScript*.

This book explains frontend development using HTML, CSS, and JavaScript. It discusses DOM manipulation, event-driven programming, state management using in-memory variables and browser storage, and modular page structure. The architectural principles helped in developing dynamic and responsive frontend modules for the Restaurant E-Commerce Website.

[6] K. Chodorow, *JSON file storage: The Definitive Guide*.

This book provides detailed insights into JSON file storage and data management systems. It explains JSON-based document storage, schema flexibility, and scalability. The concepts helped in designing efficient JSON data files and relationships for the Restaurant E-Commerce Website.

[7] I. Sommerville, *Software Engineering*.

This book explains software engineering methodologies, SDLC models, system planning, testing, and project management concepts. The techniques helped in understanding project planning, requirement analysis, and system design methodologies used during development.

CONFERENCE PAPER

[8] S. Mehta and P. Jain, "Scalable Node.js/Express and Vanilla Frontend Stack Applications for E-Commerce Platforms," in *Proceedings of the International Conference on Emerging Technologies in Computing*, 2023, pp. 112–118.

This conference paper presents the use of the Node.js/Express with vanilla frontend stack in building scalable and high-performance web applications. The authors explain how REST APIs improve modularity and integration capabilities. The concepts helped justify the use of modular APIs and a JavaScript-based web stack for developing the Restaurant E-Commerce Website.

WEBSITE REFERENCES

[9] MDN Web Docs: HTML, CSS, JavaScript and Web APIs. [Online]. Available: <https://developer.mozilla.org>

The official HTML, CSS, and JavaScript documentation provides guidance on DOM manipulation, event handling, state management, Fetch API usage, and frontend optimization. Used extensively while designing dynamic user interfaces and JavaScript helpers.

[10] Node.js Official Documentation. [Online]. Available: <https://nodejs.org>

The Node.js documentation explains server-side JavaScript execution, asynchronous programming, event-driven architecture, and backend runtime management.

[11] Node.js File System Documentation and JSON Data Handling. [Online]. Available: <https://nodejs.org/api/fs.html>

The Node.js fs module documentation provides guidance on reading and writing files, JSON data persistence, and synchronous/asynchronous file operations. Used while implementing the JSON file-based data storage layer.

[12] MDN Web Docs (Mozilla Developer Network). [Online]. Available: <https://developer.mozilla.org>

MDN Web Docs provide extensive learning material on HTML, CSS, JavaScript, and modern web APIs. Referred for frontend development concepts, responsive design practices, and debugging techniques.

CHAPTER 3

PLANNING OF THE PROJECT

3.1 Objectives of the Project

3.1.1 Functional Objectives

- To develop a complete online food ordering system
- To enable users to register, login, and manage their accounts securely
- To allow users to browse menu items categorized efficiently
- To implement a dynamic cart system for adding and removing items
- To provide a smooth and efficient order placement process
- To develop an admin panel for managing menu items and orders

3.1.2 Technical Objectives

- To design a three-tier architecture (Frontend, Backend, Data Storage)
- To implement RESTful APIs for communication between client and server
- To develop a responsive frontend using HTML, CSS, and JavaScript
- To use Node.js and Express.js for backend development
- To design JSON data storage using JSON file storage
- To ensure modular and maintainable code structure

3.1.3 Performance Objectives

- Reduce API response time to less than 2 seconds
- Optimize frontend rendering for smooth user experience
- Handle multiple users simultaneously without lag
- Minimize unnecessary API calls

3.1.4 Security Objectives

- Implement secure password hashing using bcrypt
- Use JWT (JSON Web Tokens) for authentication
- Restrict admin access using role-based authorization
- Prevent unauthorized API access

3.1.5 Usability Objectives

- Provide a simple and intuitive user interface
- Ensure easy navigation across pages
- Maintain consistency in UI design
- Improve overall user experience

3.2 Planned Steps to Achieve Above Objectives

3.2.1 Development Model Used

The project uses an **Iterative Development Model**, where the system is developed in phases, each phase is tested before moving forward, and improvements are made continuously. Advantages: early detection of errors, continuous improvement, flexible to changes, faster development cycles.

3.3 System Architecture Planning

The system is designed using a three-tier architecture, which ensures separation of concerns and scalability.

1. **Presentation Layer (Frontend)** – Built using HTML5, CSS3, and JavaScript; handles user interaction and displays data.
2. **Application Layer (Backend)** – Built using Node.js and Express.js; processes business logic and handles API requests.
3. **Data Layer (Data Storage)** – JSON files are used; stores all application data and ensures fast data retrieval.

Advantages: Scalability, Maintainability, Clear separation of responsibilities.

3.2.2 Detailed Stepwise Planning

Step	Activity	Details
1	Requirement Analysis	Identify users (Customer, Admin); define use cases (Login/Register, Browse Menu, Place Order); identify constraints
2	System Design	Define architecture; plan data flow; identify modules
3	Data File Design	Create JSON files: Users, Menu Items, Orders, Cart; define relationships
4	API Design	POST /api/auth/register, POST /api/auth/login, GET /api/menu, POST /api/orders
5	Frontend Development	Build Navbar, Menu list, Cart; integrate Fetch API; ensure responsive design
6	Backend Development	Setup Express server; implement routes; configure JSON file helpers; implement middleware
7	Integration	Connect frontend with backend; test API responses; handle errors
8	Testing	Postman API testing; UI testing; debug errors
9	Deployment	Run frontend and backend through Express server; monitor system

3.2.3 Risk Analysis and Mitigation

Risk	Description	Mitigation
Data Loss	Loss of user/order data	Data Storage backups
Security Breach	Unauthorized access	Encryption, authentication
Server Failure	Backend crash	Error handling, monitoring
Performance Issues	Slow response	Optimization
Bugs	System errors	Thorough testing

3.2.4 Project Timeline (6-Month Plan)

Month	Phase	Activities
1	Planning	Requirement analysis, Literature survey
2	Design	System architecture, Data Storage design
3	Backend Development	API development, Authentication system
4	Frontend Development	UI design, API integration
5	Testing	Debugging, Performance optimization
6	Deployment	Final local server testing, Documentation

3.2.5 Feasibility Analysis

- **Technical Feasibility:** Node.js/Express with vanilla frontend stack is suitable; all required tools are available.
- **Economic Feasibility:** Low-cost development using open-source tools.
- **Operational Feasibility:** Easy to use system requiring minimal training.

CHAPTER 4

HARDWARE AND SOFTWARE REQUIREMENTS FOR THE PROJECT

4.1 Hardware Requirements

Component	Minimum	Recommended
Processor	Intel i3 or equivalent	Intel i5/i7
RAM	4 GB	8 GB or higher
Storage	10 GB free space	20 GB or more (SSD preferred)
Network	Stable internet connection	Broadband
End-User Device	Smartphone / Laptop	Modern browser required

4.2 Software Requirements

4.2.1 Frontend Technologies

- **HTML5:** Provides structure of web pages; defines elements like forms, buttons, inputs.
- **CSS3:** Used for styling and layout; enables responsive design.
- **JavaScript (ES6+):** Adds interactivity; handles client-side logic; manages dynamic DOM updates and event-driven behavior.
- **Vanilla JavaScript (ES6+ Modules):** Core scripting language for frontend interactivity; handles all DOM manipulation and dynamic content rendering; manages client-side state via localStorage and in-memory variables.

Advantages: No external framework dependency; fast and direct DOM rendering; full control over event handling and application state.

4.2.2 Backend Technologies

- **Node.js:** JavaScript runtime environment; executes server-side code; event-driven architecture with non-blocking I/O.

- **Express.js:** Web framework for Node.js; simplifies API development; provides routing and middleware support.
- **bcryptjs:** Secure password hashing library.
- **jsonwebtoken:** JWT token generation and verification.
- **helmet:** Security middleware for HTTP headers.
- **express-rate-limit:** API rate limiting to prevent abuse.
- **uuid:** Unique identifier generation for records.

4.2.3 Data Storage Technology

JSON file storage is used for storing all application data as structured JSON files on the server filesystem using Node.js's built-in `fs` module. Features: stores data in JSON format; flexible schema; simple read/write operations using `fs.readFileSync` and `fs.writeFileSync`.

4.3 Tools Used for Development

Tool	Purpose
Visual Studio Code	Code editor with extensions for JavaScript/Node.js development
Git & GitHub	Version control system; code hosting and backup
Postman	API testing — send GET/POST/PUT/DELETE requests and debug responses
Chrome / Firefox	Run application; developer tools for debugging
VS Code JSON Viewer	Visualize JSON data files stored on filesystem
npm	Package manager; installs and manages project dependencies

4.4 Techniques Used for Development

4.4.1 RESTful API Architecture

The system uses REST (Representational State Transfer) architecture with stateless communication, client-server separation, and resource-based URLs. HTTP methods used: GET (retrieve), POST (create), PUT (update), DELETE (remove).

4.4.2 CRUD Operations

Create (add users/orders), Read (fetch menu items/orders), Update (modify cart/order status), Delete (remove items/orders).

4.4.3 Authentication and Authorization

Authentication verifies user identity through a secure login system. Authorization controls resource access through admin-only routes. Security techniques: password hashing (bcrypt), JWT tokens.

4.4.4 Asynchronous Programming

Node.js uses asynchronous programming (async/await, Promises) to handle multiple requests simultaneously and improve performance.

4.4.5 Section-Based Modular Architecture (JavaScript)

UI divided into logical sections with dedicated JavaScript functions. Each section has isolated event handlers and data loaders. Examples: Navbar toggle, Menu card rendering, Cart drawer management, Authentication modal.

4.4.6 State Management

JavaScript manages application state using DOM state variables (in-memory JavaScript variables) and `localStorage` for persisting cart items and authentication tokens. Ensures dynamic UI updates through targeted DOM manipulation and efficient rendering through selective element updates without full page reloads.

4.4.7 Data File Design Techniques

Use of JSON data files with flexible JSON structure and simple structured files for easy retrieval.

4.4.8 Error Handling

Try-catch blocks on all async operations; Express middleware error handling for server errors.

4.4.9 Testing Techniques

API testing using Postman; UI testing using browser developer tools; debugging using console logging.

4.4.10 Responsive Design

CSS media queries and mobile-friendly design ensure compatibility across desktop, tablet, and mobile devices.

Advantages of Selected Technology Stack

Category	Benefits
Stack	Full JavaScript stack; fast development; scalable architecture
Performance	Fast API response; efficient frontend DOM rendering
Scalability	Can handle growing users; flexible JSON records

CHAPTER 5

DESIGN OF THE PROJECT

5.1 ER/DFD/Flow Chart Diagrams

System design defines how different components of the application interact with each other. The Restaurant E-Commerce Website is designed using a modular and layered architecture, ensuring scalability, maintainability, and performance.

5.1.1 Flow Chart of the System

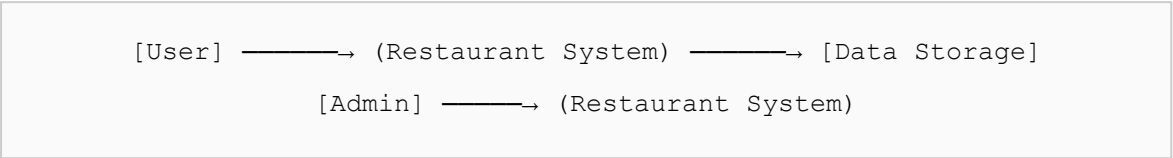


The system starts when a user accesses the website. The user must authenticate before placing an order. After browsing and adding items, the order is stored in the data storage

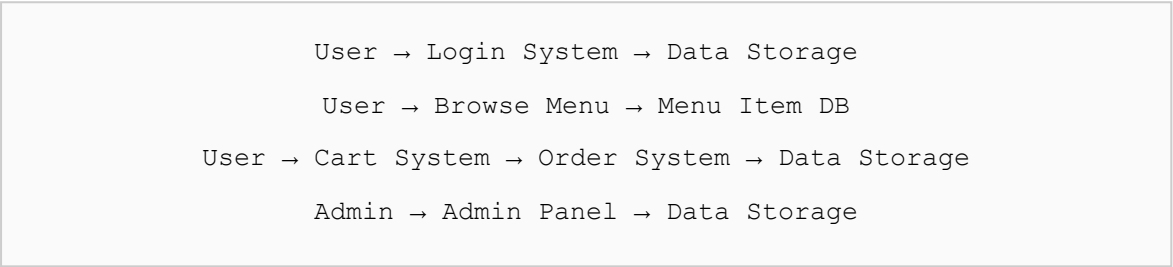
and the admin processes it.

5.1.2 Data Flow Diagram (DFD)

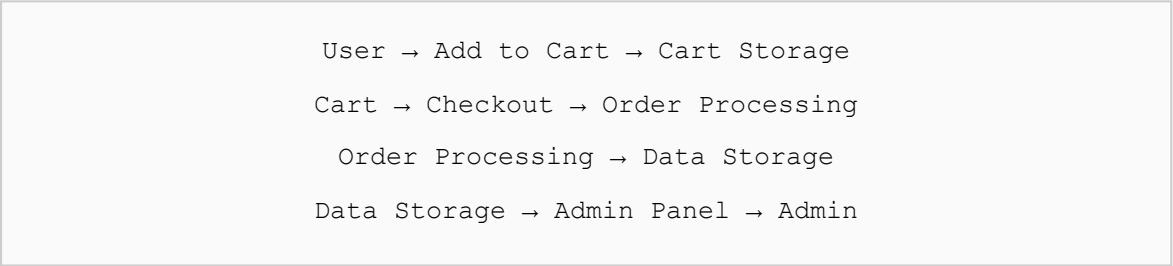
Level 0 DFD (Context Diagram)



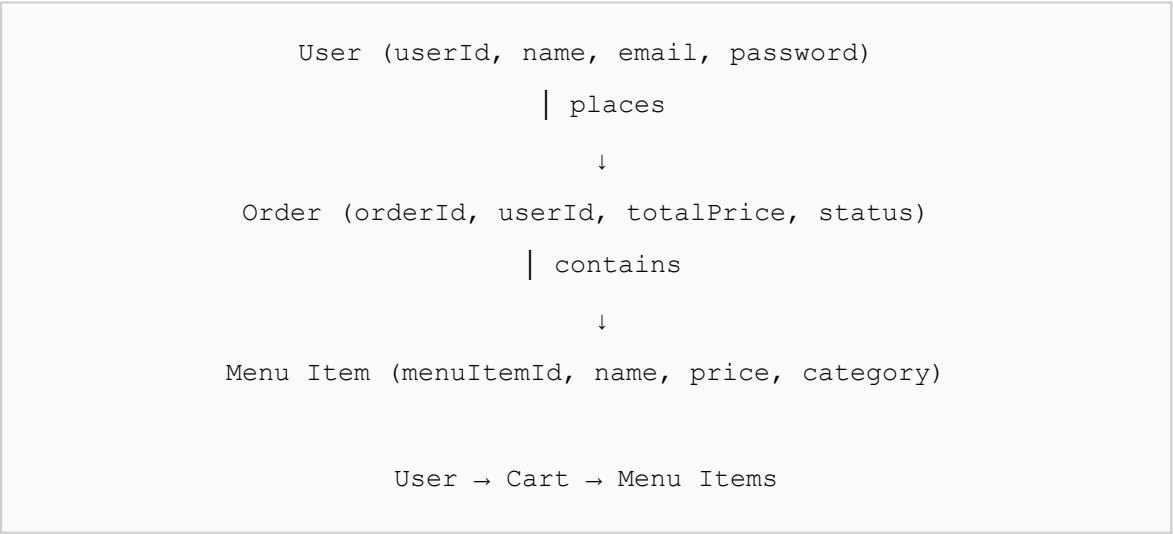
Level 1 – Admin Panel DFD



Level 1 – User Panel DFD (Detailed Order Flow)



5.1.3 ER Diagram (Entity Relationship Design)



A user can place multiple orders; each order contains multiple menu items; each user has a cart.

5.2 Class Diagram

Class	Attributes	Relationships
User	userId, name, email, password, role	One-to-Many → Order; One-to-One → Cart
Menu Item	menuItemId, name, category, price, description, badge, image	Many-to-Many ↔ Order
Order	orderId, userId, items[], totalPrice, status, paymentMethod, createdAt	Many-to-One → User
Cart	items[], quantities (stored in localStorage)	One-to-One → User

5.3 Data Storage Tables

Users Data File (`data/users.json`)

Field	Type	Description
id	String (UUID)	Unique identifier
name	String	User full name
email	String	Email address (unique)
password	String	bcrypt-hashed password
role	String	'customer' or 'admin'
createdAt	ISO String	Registration timestamp

Menu Data File (data/menu.json)

Field	Type	Description
id	String	Unique identifier
name	String	Menu item name
category	String	Category (appetizers, main-course, seafood...)
price	Number	Price in INR
description	String	Item description
image	String	Relative path to image asset
badge	String / null	Label e.g. "New", "Seasonal", "Best Seller"

Orders Data File (data/orders.json)

Field	Type	Description
id	String (UUID)	Unique order ID
userId	String	Reference to placing user
customerName / Email / Phone	String	Customer contact details
deliveryAddress	String	Delivery address
items	Array	List of ordered menu items with quantities
totalPrice	Number	Total cost
paymentMethod	String	Cash on Delivery / Card on Delivery
status	String	pending / preparing / out-for-delivery / completed / cancelled
createdAt	ISO String	Order placement timestamp

Client Cart Storage (`localStorage`)

Key	Type	Description
feastflow_cart	JSON Array	Cart items with id, name, price, image, quantity
feastflow_token	String	JWT authentication token
feastflow_user	JSON Object	Logged-in user profile

System Design Advantages & Limitations

Advantages	Limitations
Modular design — easy to maintain and extend	No real-time tracking
Scalable architecture	No payment gateway integration
Flexible JSON file storage	Single restaurant system
Fast API responses	

CHAPTER 6

IMPLEMENTATION OF THE PROJECT

6.1 Screenshots of Working Project

6.1.1 Homepage Interface of the Website

The homepage serves as the main entry point for users, featuring a visually appealing banner with high-quality food imagery, a navigation bar, and prominent call-to-action buttons ("Find a Table", "View Our Menu"). It ensures easy navigation, responsiveness, and accessibility across devices.



Figure 6.1 — Feast Flow Logo & Brand Identity

6.1.2 Menu Section of the Website

The menu section displays food items in an organized and visually appealing manner. Each item includes the name, price, description, and an image. Items are divided into categories and highlighted with labels like "Seasonal", "New", and "Best Seller". Users can add items directly to the cart from this section.



Figure 6.2 — Paneer Tikka (Appetizer) — ₹260



Figure 6.3 — Butter Chicken (Main Course) — ₹420



Figure 6.4 — Hyderabad Biryani (Main Course) — ₹360



Figure 6.5 — Shami Kebab (Appetizer) — ₹280



Figure 6.6 — Goan Fish Curry (Seafood) — ₹450

6.1.3 Upcoming Events / Blog Section

This section showcases recent updates, events, and featured content with cards containing images, dates, titles, and short descriptions. A "View Our Blog" button allows users to explore more. It helps improve user interaction and keep the website dynamic.

6.1.4 Online Reservation and Contact Section

This section allows users to book a table by filling out a reservation form with name, phone number, number of persons, date, time, and additional messages. Alongside the form, contact information (phone, location, operating hours) is displayed. Form submissions are handled via the `/api/reservations` REST endpoint and persisted to `data/reservations.json`.

6.1.5 "Why Choose Us" / Our Strength Section

This section highlights key strengths — hygienic food preparation, fresh environment, experienced chefs, and event facilities — using icons and short descriptions to build trust with customers.

6.1.6 Customer Testimonial Section

A testimonial carousel displays customer reviews with quotes, names, and profile images. The auto-advancing carousel with clickable dot navigation is implemented in vanilla JavaScript.

6.1.7 Footer Section and Newsletter Subscription

The footer provides essential restaurant information (address, contact details, operating hours), a navigation menu, and social media links. A newsletter subscription form accepts

user email and persists the subscription via the `/api/newsletter` endpoint.

6.2 Code Snippets

6.2.1 Parallax Effect JavaScript Implementation

This code implements a parallax scrolling effect based on mouse position. Elements with the `data-parallax-item` attribute are moved at different speeds based on `data-parallax-speed`, creating a layered depth effect on the homepage.

```
const parallaxItems = document.querySelectorAll("[data-parallax-item]");
let x, y;

window.addEventListener("mousemove", function (event) {
  x = (event.clientX / window.innerWidth * 10) - 5;
  y = (event.clientY / window.innerHeight * 10) - 5;
  x = x - (x * 2);
  y = y - (y * 2);

  for (let i = 0, len = parallaxItems.length; i < len; i++) {
    const speedX = x * Number(parallaxItems[i].dataset.parallaxSpeed);
    const speedY = y * Number(parallaxItems[i].dataset.parallaxSpeed);
    parallaxItems[i].style.transform =
      `translate3d(${speedX}px, ${speedY}px, 0px)`;
  }
});
```

6.2.2 Navigation Bar Toggle Functionality

This code handles mobile navbar toggling. Clicking any `[data-nav-toggler]` element adds or removes the `active` class on the navbar and overlay, providing a smooth open/close animation suitable for small screens.

```
const navbar      = document.querySelector("[data-navbar]");
const navTogglers = document.querySelectorAll("[data-nav-toggler]");
const overlay     = document.querySelector("[data-overlay]");

const toggleNavbar = function () {
  navbar.classList.toggle("active");
  overlay.classList.toggle("active");
  document.body.classList.toggle("nav-active");
}

addEventListener(navTogglers, "click", toggleNavbar);
```

6.2.3 Hero Slider Functionality

The hero slider cycles through banner slides automatically every 7 seconds. The `slideNext` / `slidePrev` functions manage active class toggling across the slide items. A typing animation is triggered for each slide's subtitle using the `typeSubtitle` function.

```
let currentSlidePos = 0;
let lastActiveSliderItem = heroSliderItems[0];

const updateSliderPos = function () {
  lastActiveSliderItem.classList.remove("active");
  heroSliderItems[currentSlidePos].classList.add("active");
  lastActiveSliderItem = heroSliderItems[currentSlidePos];
  resetProgress();
  typeSubtitle(heroSliderItems[currentSlidePos]);
}

const slideNext = function () {
  if (currentSlidePos >= heroSliderItems.length - 1) currentSlidePos = 0;
  else currentSlidePos++;
  updateSliderPos();
}

const slidePrev = function () {
  if (currentSlidePos <= 0) currentSlidePos = heroSliderItems.length - 1;
  else currentSlidePos--;
  updateSliderPos();
}

if (heroSliderNextBtn) heroSliderNextBtn.addEventListener("click",
slideNext);
if (heroSliderPrevBtn) heroSliderPrevBtn.addEventListener("click",
slidePrev);

const startAutoSlide = () => { autoSlideInterval = setInterval(slideNext,
7000); }
window.addEventListener("load", () => { if (heroSliderItems.length > 0)
startAutoSlide(); });
```

6.2.4 Online Reservation Form Structure (HTML Implementation)

The reservation form uses the `wireForm` utility which intercepts submit, validates required fields, serializes the form to JSON, and sends a POST to `/api/reservations`. The server persists the booking to `data/reservations.json` and returns a success toast.

```
wireForm("reservation-form", "/api/reservations",
  "Reservation created successfully!");
wireForm("newsletter-form", "/api/newsletter",
  "Subscribed to newsletter successfully!");
wireForm("contact-form", "/api/contact",
  "Message sent successfully!");

// wireForm internally does:
const response = await fetch(endpoint, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(data)
});
```

6.2.5 Express Server Setup and API Routing

The `server.js` file configures the Express application with Helmet (HTTP security headers), CORS, API rate limiting (100 requests per 15 minutes), and registers all route modules. The server auto-increments the port if the default is already occupied.

```

'use strict';
const express = require('express');
const cors    = require('cors');
const helmet   = require('helmet');
const rateLimit = require('express-rate-limit');

const app = express();
const PORT = process.env.PORT || 3000;

app.use(helmet({ contentSecurityPolicy: false }));
app.use(cors());
app.use(rateLimit({ windowMs: 15 * 60 * 1000, max: 100 }));
app.use(express.json());
app.use(express.static(path.join(__dirname)));

app.use('/api/reservations', reservationsRouter);
app.use('/api/newsletter', newsletterRouter);
app.use('/api/contact', contactRouter);
app.use('/api/menu', menuRouter);
app.use('/api/auth', authRouter.router);
app.use('/api/orders', ordersRouter);

function startServer(port) {
  const server = app.listen(port, () =>
    console.log(`Feast Flow running at http://localhost:${port}`)
  );
  server.on('error', err => {
    if (err.code === 'EADDRINUSE') startServer(port + 1);
  });
}
authRouter.seedAdmin();
startServer(PORT);

```

6.2.6 JSON File-Based Data Helpers for Menu Management

All route files follow the same `readData` / `writeData` pattern using Node.js's built-in `fs` module. The menu GET endpoint supports an optional `?category=` query parameter for filtered browsing.

```

const DATA_FILE = path.join(__dirname, '..', 'data', 'menu.json');

function readData() {
  try {
    const raw = fs.readFileSync(DATA_FILE, 'utf-8');
    return JSON.parse(raw);
  } catch { return []; }
}

function writeData(data) {
  fs.writeFileSync(DATA_FILE, JSON.stringify(data, null, 2));
}

// GET /api/menu - optional ?category= filter
router.get('/', (req, res) => {
  let menu = readData();
  if (req.query.category) {
    menu = menu.filter(item =>
      item.category.toLowerCase() === req.query.category.toLowerCase()
    );
  }
  res.json({ success: true, data: menu, total: menu.length });
});

```

6.2.7 JWT-Based User Authentication Route

Passwords are hashed with `bcryptjs` (salt rounds = 10) before storage. On successful login or registration, a 24-hour JWT signed with the server secret is returned to the client, which stores it in `localStorage` for subsequent authenticated requests.


```

// POST /api/auth/register
router.post('/register', (req, res) => {
  const { name, email, password } = req.body;
  if (!password || password.length < 6)
    return res.status(400).json({ error: 'Password min 6 characters.' });

  const normalizedEmail = email.trim().toLowerCase();
  const users = readData();
  if (users.some(u => u.email === normalizedEmail))
    return res.status(400).json({ error: 'Email already exists.' });

  const hashedPassword = bcrypt.hashSync(password, bcrypt.genSaltSync(10));
  const newUser = { id: uuidv4(), name: name.trim(), email: normalizedEmail,
    password: hashedPassword, role: 'customer',
    createdAt: new Date().toISOString() };
  users.push(newUser); writeData(users);
  const token = jwt.sign({ id: newUser.id, role: newUser.role },
    JWT_SECRET, { expiresIn: '24h' });
  res.status(201).json({ success: true, token });
});

// POST /api/auth/login
router.post('/login', (req, res) => {
  const { email, password } = req.body;
  const users = readData();
  const user = users.find(u => u.email === email.trim().toLowerCase());
  if (!user || !bcrypt.compareSync(password, user.password))
    return res.status(400).json({ error: 'Invalid email or password.' });

  const token = jwt.sign({ id: user.id, name: user.name,
    email: user.email, role: user.role },
    JWT_SECRET, { expiresIn: '24h' });
  res.json({ success: true, token,
    user: { id: user.id, name: user.name, email: user.email, role:
user.role } });
});

```

6.2.8 Client-Side Cart Management

The cart is persisted in `localStorage` as a JSON array. `addToCart` increments the quantity if the item already exists, otherwise pushes a new entry. Every mutation calls `saveCart` which triggers `updateCartUI` to re-render the drawer.

```

const getCart = () => {
  try { return JSON.parse(localStorage.getItem("feastflow_cart")) || []; }
  catch { return []; }
};

const saveCart = (cart) => {
  localStorage.setItem("feastflow_cart", JSON.stringify(cart));
  updateCartUI();
};

const addToCart = (id, name, price, image) => {
  const cart = getCart();
  const existing = cart.find(item => item.id === id);
  if (existing) existing.quantity += 1;
  else cart.push({ id, name, price: Number(price), image, quantity: 1 });
  saveCart(cart);
  showToast(`Added ${name} to cart!`, "success");
};

const updateCartQuantity = (id, amount) => {
  let cart = getCart();
  const item = cart.find(item => item.id === id);
  if (!item) return;
  item.quantity += amount;
  if (item.quantity <= 0) cart = cart.filter(item => item.id !== id);
  saveCart(cart);
};

```

6.2.9 Order Placement Route (Backend)

The `POST /api/orders` route requires a valid JWT. It validates the cart payload, calculates the total price server-side (to prevent client-side tampering), and persists the order to `data/orders.json`. Admins can update status via `PATCH /api/orders/:id`.

```

router.post('/', authenticateJWT, (req, res) => {
  const { phone, address, items, paymentMethod, notes } = req.body;
  if (!phone || !address)
    return res.status(400).json({ error: 'Phone and address required.' });
  if (!items || !items.length)
    return res.status(400).json({ error: 'Cart is empty.' });

  let totalPrice = 0;
  for (const item of items)
    totalPrice += Number(item.price) * Number(item.quantity);

  const newOrder = {
    id: uuidv4(), userId: req.user.id, customerName: req.user.name,
    customerEmail: req.user.email, customerPhone: phone.trim(),
    deliveryAddress: address.trim(),
    items: items.map(i => ({ name: i.name, price: Number(i.price),
      quantity: Number(i.quantity) })),
    totalPrice: Number(totalPrice.toFixed(2)),
    paymentMethod: paymentMethod || 'Cash on Delivery',
    notes: notes ? notes.trim() : '', status: 'pending',
    createdAt: new Date().toISOString()
  };
  const orders = readData(); orders.push(newOrder); writeData(orders);
  res.status(201).json({ success: true, order: newOrder });
});

// PATCH /api/orders/:id - Admin only
router.patch('/:id', authenticateJWT, isAdmin, (req, res) => {
  const { status } = req.body;
  const validStatuses = ['pending', 'preparing',
    'out-for-delivery', 'completed', 'cancelled'];
  if (!validStatuses.includes(status))
    return res.status(400).json({ error: 'Invalid status.' });

  const orders = readData();
  const order = orders.find(o => o.id === req.params.id);
  if (!order) return res.status(404).json({ error: 'Order not found.' });
  order.status = status; writeData(orders);
  res.json({ success: true, order });
});

```

6.2.10 Admin Dashboard — Orders Table with Status Management

The admin orders tab fetches all orders using the admin JWT via `GET /api/orders/all`, computes live metrics (total, pending, in-progress, revenue), and renders an editable status `<select>` per row that calls `updateOrderStatus` on change.

```

async function fetchOrders() {
  const token = getAdminToken();
  if (!token) { showAdminLoginModal(); return; }
  const response = await fetch('/api/orders/all', {
    headers: { 'Authorization': 'Bearer ' + token }
  });
  if (response.ok) {
    const result = await response.json();
    ordersData = result.data;
    applyOrdersFilter();
    renderOrdersMetrics(ordersData);
  }
}

function renderOrdersMetrics(data) {
  const revenue = data
    .filter(o => o.status !== 'cancelled')
    .reduce((sum, o) => sum + o.totalPrice, 0);
  document.getElementById('metric-total').textContent = data.length;
  document.getElementById('metric-pending').textContent =
    data.filter(o => o.status === 'pending').length;
  document.getElementById('metric-revenue').textContent =
    formatCurrency(revenue);
}

async function updateOrderStatus(id, status) {
  const token = getAdminToken();
  const response = await fetch(`/api/orders/${id}`, {
    method: 'PATCH',
    headers: { 'Content-Type': 'application/json',
      'Authorization': 'Bearer ' + token },
    body: JSON.stringify({ status })
  });
  if (response.ok) { showToast(`Status → ${status}`, 'success');
  fetchOrders(); }
}

```

6.2.11 Dynamic Menu Rendering with Add to Cart Button

On page load, `loadMenu` fetches `GET /api/menu` and replaces the static HTML fallback with dynamically generated menu cards. Each card's "Add to Cart" button carries `data-` attributes that the cart event delegation handler reads to add the item.

```

const loadMenu = async () => {
  if (!menuGrid) return;
  const response = await fetch("/api/menu");
  if (!response.ok) return;
  const { data: menuItems } = await response.json();
  if (!menuItems.length) return;
  menuGrid.innerHTML = "";
  menuItems.forEach((item, index) => {
    const imagePath = normalizeAssetPath(item.image);
    const li = document.createElement("li");
    li.setAttribute("data-reveal", "fade-up");
    li.style.animationDelay = `${index * 50}ms`;
    li.innerHTML = `
      <div class="menu-card">
        <figure class="card-banner">
          
        </figure>
        <div>
          <div class="title-wrapper">
            <h3 class="title-3">${item.name}</h3>
            ${item.badge ? `<span class="badge">${item.badge}</span>` : ""}
            <span class="title-2">${formatCurrency(item.price)}</span>
          </div>
          <p class="card-text">${item.description}</p>
          <button class="add-to-cart-btn"
            data-id="${item.id}" data-name="${item.name}"
            data-price="${item.price}" data-image="${imagePath}">
            Add to Cart
          </button>
        </div>
      </div>`;
    menuGrid.appendChild(li);
    revealObserver.observe(li);
  });
};

window.addEventListener("load", loadMenu);

```

6.2.12 Cart Drawer Scrollable Layout (CSS)

The fixed-position cart drawer slides in from the right. `.cart-drawer-body` uses `flex: 1 1 auto` with `overflow-y: auto` so only the item list scrolls while the header and checkout footer remain fixed in view.

```

.cart-drawer {
  position: fixed; top: 0; right: -420px;
  width: 100%; max-width: 400px;
  height: 100vh; height: 100dvh;
  background: var(--smoky-black-1);
  box-shadow: -10px 0 30px rgba(0,0,0,.5);
  z-index: 30; display: flex; flex-direction: column;
  transition: 0.4s cubic-bezier(0.22, 1, 0.36, 1);
}
.cart-drawer.active { right: 0; }

.cart-drawer-overlay {
  position: fixed; inset: 0;
  background: rgba(0,0,0,.7);
  backdrop-filter: blur(5px);
  opacity: 0; visibility: hidden; z-index: 25;
  transition: var(--transition-1);
}
.cart-drawer-overlay.active { opacity: 1; visibility: visible; }

.cart-drawer-body {
  flex: 1 1 auto; min-height: 0; padding: 24px;
  overflow-x: hidden; overflow-y: auto;
  overscroll-behavior: contain; gap: 20px;
  -webkit-overflow-scrolling: touch;
}

```

6.2.13 Currency Formatter and Asset Path Normalizer

These two utility functions are shared across both `script.js` and `admin.js`. `formatCurrency` uses the browser's `Intl.NumberFormat` API to render prices in Indian Rupees. `normalizeAssetPath` ensures image paths stored in JSON always resolve correctly regardless of whether they start with `/`, `./`, or a full URL.

```
const CURRENCY_FORMATTER = new Intl.NumberFormat("en-IN", {
  style: "currency", currency: "INR", maximumFractionDigits: 0
});

const formatCurrency = (amount) =>
  CURRENCY_FORMATTER.format(Number(amount) || 0);

const normalizeAssetPath = (path) => {
  if (!path) return "./assets/images/menu-1.jpg";
  if (/^(https?:)?\:\/\/.test(path) || path.startsWith("data:"))
    return path;
  if (path.startsWith("/")) return `.${path}`;
  return path;
};
```

CHAPTER 7

CONCLUSIONS / FUTURE SCOPE OF THE PROJECT

7.1 Conclusions of the Project

The Restaurant E-Commerce Website project was successfully developed to provide a modern, user-friendly, and efficient online platform for restaurant management and food ordering services. The main objective of the project was to simplify the process of browsing menus, placing food orders, managing customer information, and handling restaurant operations digitally. The system helps bridge the gap between customers and restaurants by offering a convenient and time-saving solution.

The project integrates various important functionalities such as user registration and login, menu display, food categorization, cart management, online ordering, and order tracking. An admin panel was also implemented to allow restaurant administrators to manage food items, customer orders, prices, and other essential operations effectively. The website was designed with a responsive interface so that users can access it easily through desktops, tablets, and mobile devices.

During the development of the project, modern web technologies and data management concepts were applied successfully. The project improved practical understanding of frontend development, backend integration, JSON data-file integration, and user interface design. Security and data management techniques were also considered to ensure proper handling of customer information and transactions.

The outcomes demonstrate that the system is capable of reducing manual work, improving order accuracy, and enhancing customer satisfaction. Overall, the Restaurant E-Commerce Website achieved its intended goals and proved to be a reliable, scalable, and effective solution for online food ordering and restaurant management.

7.2 Results in Tabular Form

TABLE 1: Comparison Between Existing System and Proposed System

Parameter	Existing System (Traditional)	Proposed System (E-Commerce)
Order Placement	Manual (phone/in-person)	Website-based
Time Consumption	High	Low
Menu Accessibility	Limited (physical menu)	Available anytime, anywhere
Order Accuracy	Prone to human errors	High accuracy
Payment Method	Mostly cash	Cash / Card on Delivery
Record Management	Manual records	Automated JSON file records
Customer Convenience	Limited	High
Scalability	Difficult	Easily scalable

TABLE 2: Functional Results of the System

Module	Expected Outcome	Actual Result
User Registration	Easy account creation	Successfully implemented
Login System	Secure authentication	Working accurately (JWT + bcrypt)
Menu Display	Organized food categories	Clear and user-friendly interface
Cart System	Add/remove items easily	Smooth localStorage-based functionality
Order Placement	Quick and efficient ordering	Successfully achieved
Admin Panel	Manage items and orders	Fully functional with 5 tabs
Data File Handling	Store data in JSON files	Reliable local JSON data storage

TABLE 3: Performance Evaluation

Criteria	Result
System Efficiency	High
User Satisfaction	High
Response Time	Fast (< 2 seconds)
Error Rate	Minimal
Reliability	Strong

7.3 Future Scope of Project

1. **Mobile Application Development:** Develop a dedicated Android/iOS app to improve accessibility.
2. **Online Payment Integration:** Add UPI, credit/debit card, and digital wallet support for seamless transactions.
3. **Real-Time Order Tracking System:** Allow customers to track food orders in real time for better transparency.
4. **AI-Based Recommendation System:** Suggest food items based on user preferences and previous order history.
5. **Multi-Restaurant Support:** Expand into a multi-vendor platform where multiple restaurants can register and offer services.

CHAPTER 8

ABOUT THE ORGANISATION

PG Tech Solutions is a leading IT training and software development company established to provide industry-oriented technical education and advanced digital solutions. They specialize in a wide range of services including Web Development, Mobile Application Development, Digital Marketing, Search Engine Optimization (SEO), Software Development, UI/UX Designing, Cloud Computing, Cyber Security, Artificial Intelligence, Machine Learning, Data Science, Graphic Designing, Video Editing, and IT Consulting Services. PG Tech Solutions also offers Industrial Training, Internship Programs, Corporate Training, Placement Assistance, and Job-Oriented Courses for students and professionals.

PG Tech Solutions provides complete IT and software solutions supporting the entire business process from planning and consulting to development, deployment, maintenance, testing, and technical support. With experienced trainers and developers, the company focuses on delivering practical learning, live project exposure, affordable training programs, and career-oriented technical skills to students and businesses. Their goal is to bridge the gap between academic knowledge and real industry requirements through hands-on project-based learning and professional mentorship.

Additionally, PG Tech Solutions provides training programs including 6 Weeks/6 Months Industrial Training, Internship-Based Training, Project-Based Training, Corporate Training, and Job-Oriented Courses covering major IT technologies such as Full Stack Development (MEAN/MERN), JavaScript, Angular, Node.js, Vue.js, Python, AI, Machine Learning, Data Science, Cloud Computing, Cyber Security, Flutter, Android Development, Java, PHP, Laravel, WordPress, Digital Marketing, SEO, UI/UX Designing, and many more.

The company emphasizes practical implementation, real-time projects, certification programs, interview preparation, and placement support to help students and professionals build successful careers in the IT industry.

Phone	+91-9815595408, +91-7717313727
E-Mail	gatiprerna@gmail.com
Website	www.pgtechsolutions.com
Office (1)	Phase 8B, Industrial Area, Sector 74, Mohali, Punjab
Office (2)	Sector 34A, Chandigarh, India

CHAPTER 9

REFERENCES

JOURNAL

1. A. Kumar and S. Gupta, "Secure Authentication Systems in Web Applications," *International Journal of Computer Applications*, vol. 183, no. 12, pp. 25–31, 2021.
2. R. Patel and M. Shah, "Scalable Web Application Design for E-Commerce Platforms," *IEEE Access*, vol. 10, pp. 112–118, 2022.

ONLINE JOURNAL

3. M. Verma and P. Arora. (2023). Modern Node.js/Express and Vanilla Frontend Stack Development Techniques for E-Commerce Applications. *Journal of Web Engineering*. [Online]. Available: <https://www.sciencedirect.com>
4. S. Gupta and R. Mehta. (2022). Secure Authentication and Authorization in Full Stack Web Applications. *International Journal of Advanced Computer Science*. [Online]. Available: <https://ieeexplore.ieee.org>

BOOKS

5. U.K. Roy, *Web Technologies*. Oxford University Press.
6. D. Herron, *Node.js Web Development*. Packt Publishing.
7. A. Banks and E. Porcello, *Learning JavaScript*. O'Reilly Media.
8. I. Sommerville, *Software Engineering*, 10th ed., Pearson Education.

CONFERENCE PAPER

9. S. Mehta and P. Jain, "Scalable Node.js/Express and Vanilla Frontend Stack Applications for E-Commerce Platforms," in *Proceedings of the International Conference on Emerging Technologies in Computing*, 2023, pp. 112–118.

WEBSITE REFERENCES

10. MDN Web Docs: HTML, CSS, JavaScript and Web APIs. [Online]. Available: <https://developer.mozilla.org>
11. Node.js Official Documentation. [Online]. Available: <https://nodejs.org>

12. Express.js Documentation. [Online]. Available: <https://expressjs.com>
13. Node.js File System (fs) Module Documentation. [Online]. Available: <https://nodejs.org/api/fs.html>
14. MDN Web Docs (Mozilla Developer Network). [Online]. Available: <https://developer.mozilla.org>